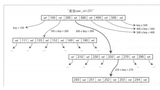
数据存储与检索

- **背景**
  - 数据库本质就是保存数据+返回查询数据
  - 了解数据库如何存储输入的数据和如何找到数据有助于我们对数据库进行调优
- **数据库的核心：数据结构**
  - 索引
    - 为什么需要索引：解决查询效率
    - 没有索引可能需要全表扫描
    - 索引的本质是用额外的存储空间换取快速查找的时间
  - 哈希索引
    - **通过哈希函数将索引键（key）映射为固定长度的哈希值，通过哈希值快速定位数据在存储中的位置**
    - 优点
      - 查询速度快，无哈希冲突时时间复杂度是 O(1)
      - 结构简单，只需存储key的哈希值和数据地址
      - 追加式写入是顺序写，比随机式写入快很多
    - 缺点
      - 不支持范围查询/排序 —— 哈希值是随机分布的，没有办法通过哈希值进行排序
      - 不支持前缀匹配
      - 哈希冲突影响性能
    - Bitcask存储引擎
      - 实际应用
        - 使用追加式日志存储数据
        - 提供高性能的读和写，适合每个键的值频繁更新的场景
        - 所有的key要加载在内存中，value数据可以通过一次磁盘寻址加载到内存中
      - 段压缩与合并
        - 解决的问题：避免追加式的日志用尽磁盘空间
        - 将日志分成一定大小的段，当文件达到一定大小时关闭然后将后续写入新的段文件中，然后再这些段文件上执行压缩（丢弃重复的键，保留键的最新值）

      - 额外的细节考虑
        - 日志格式 —— csv不是最佳文件格式，二进制格式才是最佳的
        - 删除记录 —— 删除键值时需要在数据文件中追加墓碑标记，当合并日志段时，若发现墓碑标记则丢弃这个键的所有值
        - 崩溃恢复
          - 数据库重新启动时，内存中的hash map会丢失
          - 可以全盘读取段文件，获取每个段的hash map，但是这很慢
          - 可以将每个段的hash map快照存在磁盘中
        - 值校验 —— 丢弃损坏的写入
        - 并发控制 —— 由于日志是追加式写入，因此只有一个写线程，但是可以有多个读线程
  - SSTables和LSM-Tree
    - SSTables
      - 核心改进：key在段内**有序存储**
      - 相比哈希索引的优势
        - 支持范围查询
        - 合并段文件更高效，类似于归并排序
        - 稀疏索引

不需要为每个键维护索引

        - 索引指向每N KB的起始键
      - 构建和维护SSTables —— 如何保证写入的数据有序？
        - 内存排序+定期刷盘
        - 红黑树、AVL树是优秀的内存排序数据结构
    - LSM-Tree
      - 基于合并和压缩排序文件原理的存储引擎 (Log-Structured Merge-Tree)
      - 工作流程
        - 写入时，将其添加到内存中的平衡树数据结构中
        - 当内存表大于某个阈值时，将其作为SSTable文件写入磁盘
        - 处理读请求时，首先在内存表中查找键，然后是最新的磁盘段文件，接下来是次新的磁盘段文件
        - 后台进程周期性执行段合并和压缩过程
        - 磁盘中保留每次写入的追加式日志文件，用于崩溃恢复内存表；当刷盘时可以丢弃相应的日志文件
      - 性能优化 —— 查询不存在的键时需要扫描内存表以及所有磁盘中的SSTable段文件，性能差 —— 使用布隆过滤器
      - 优点
        - 写性能极佳 —— 顺序写
        - 写入延迟低且稳定
        - 空间回收简单
          - 没有碎片
          - 段压缩自动整理
        - 批量写入友好
          - 一次刷盘写入大量数据
          - 顺序I/O最优
        - 天然支持快照
        - 并发友好（读取写入均无锁）
      - 缺点
        - 读放大严重
          - 查询需要查找各个level的段文件
          - 使用布隆过滤器也不能完全解决问题
        - 写放大巨大
        - 空间放大
          - 存在多版本数据
          - 删除数据没有及时清理
        - 范围查询复杂 —— 需要合并多层段数据
      - 应用场景
        - 写多读少
        - 批量写入
        - 时序数据
        - 日志存储
        - 追加操作为主
        - 可容忍读延迟
        - 需要高写吞吐
        - 应用
          - 监控系统
          - 日志分析
          - IOT数据采集
          - 用户行为追踪
          - 消息队列
          - 区块链
  - B-Trees
    - **自平衡的多路搜索树，通过减少树的高度+多路分支让每次查询的磁盘访问次数最小化**
    - 数据结构
      - 写操作使用新数据覆盖磁盘上的旧页，不会改变页的磁盘存储位置
      - 将数据库分解成固定大小的块或页，传统上为4 KB
      - 每个页面都可以使用地址或位置进行标识，类似于指针指向磁盘地址



      - 一个页包含的子页引用数量称为分支因子，通常为几百
    - 保证B-Tree的可靠性
      - 使用预写日志(WAL)让数据库能从崩溃中恢复
        - B-Tree原地覆盖以及插入操作中崩溃会导致索引被破坏
        - B-Tree的修改先追加式写入WAL然后再修改树本身的页
      - 使用锁存器保护树的数据结构，防止并发时看到不一致的B-Tree
    - 优化B-Tree
      - 使用写时复制代替覆盖页和维护WAL
      - 保存键的缩略信息而不是完整的键
      - 让相邻子页可以按顺序保存在磁盘上，从而提高磁盘I/O
      - 添加额外的指针到树中
    - 优点
      - 读性能优秀
      - 空间效率高
        - 相较于LSM-Tree，没有重复数据（LSM存在多版本数据）
        - 存储开销小
      - 查询范围友好
      - 性能稳定
    - 缺点
      - 写性能差
        - 随机写I/O
        - 读-修改-写开销
        - 节点分裂开销
      - 写放大
        - 页面重写
        - 节点分裂
      - 碎片化问题
      - 并发写入困难
        - 使用锁保护
        - 使用MVCC控制
      - 批量加载效率低
    - 应用场景
      - 读多写少（读写比 > 8:2）
      - 点查询为主
      - 需要范围查询
      - 需要ACID事务
      - 延迟敏感
      - 数据更新频繁
      - 应用
        - 银行系统
        - ERP系统
        - 电商订单系统
        - 库存管理
        - 用户账户系统
  - 其他索引结构
    - 二级索引 —— 键不唯一
    - 在索引中存储值
      - 聚簇索引 —— 数据按索引键的顺序物理存储，索引的叶节点就是完整的数据行
      - 非聚簇索引 —— 索引与数据分离，叶节点存储主键或行指针
      - 覆盖索引 —— 索引包含查询所需的所有列，无需回表
    - 多列索引
    - 全文搜索和模糊索引 —— 倒排索引 —— 从词到文档的映射，适合全文搜索
  - 内存数据库
    - 上述的数据结构设计都是为了适应磁盘限制（磁盘I/O慢）
    - 内存数据库可以提供基于磁盘索引难以实现的数据模型 —— Redis
    - 内存数据库架构可以扩展到支持远大于可用内存的数据集，而不会导致以磁盘为中心架构的开销
- **列存储**
  - 背景
    - 分析查询的特点：只访问少数的列，扫描大量的行
    - 行存储会读取所有列的数据，造成I/O浪费
  - 原理与优势
    - **将每列的所有值存储在一起**
    - 分析查询I/O大量减少
    - CPU缓存友好 —— 列数据连续存储，缓存命中率高
    - 天然适合向量化处理 —— SIMD指令可并行处理多个值
  - 列压缩
    - 每列数据类型相同，重复值多
    - 压缩方法
      - 位图编码
      - 游程编码
      - 字典编码
  - 优化措施
    - 写入优化 —— LSM-Tree结构
      - 写入先写入到MemStore
      - 定期刷盘到磁盘（列式文件）
      - 后台合并压缩
    - 排序优化 —— 排序键
      - 按照某列排序存储数据
      - 范围查询极快
    - 聚合优化
      - 向量化执行
        - 一次性处理多行
        - 利用SIMD指令并行计算
      - 延迟物化
        - 查询时不立即重构执行
        - 只在需要时才组合列数据
        - 减少内存拷贝
- **小结**
  - 存储引擎分类
    - OLTP（针对事务处理）
      - 日志结构派
        - 只允许追加式更新文件和删除过时文件
        - 不会修改已写入的文件
        - LSM-Tree, HBase等
      - 原地更新派
        - 将磁盘视为可以覆盖的一组固定大小的页
        - B-Tree
    - OLAP（针对分析型）